

Deploying a Dockerized Flask Application on AWS EC2 Using GitHub Actions and Amazon ECR

Introduction

Deploying modern applications in the cloud often involves several moving parts: containerization with Docker, cloud storage configuration, continuous integration/continuous deployment (CI/CD) pipelines, and cloud container registries. This article chronicles the journey of deploying a Flask-based COVID-risk prediction application on AWS, highlighting the challenges faced and solutions applied.

The goal was to deploy a Dockerized application from a GitHub repository to an EC2 instance using GitHub Actions to push images to Amazon Elastic Container Registry (ECR).

Initial Setup

1. EC2 Instance

An Ubuntu EC2 instance was created as the hosting platform. Initially, it had a root partition size of 6.8 GB, which later became a bottleneck during Docker image pulls.

2. Docker Installation

Docker needed to be installed on the EC2 instance. The steps included:

```
bash
sudo apt update
sudo apt install -y docker.io
sudo systemctl start docker
sudo systemctl enable docker
```

Once installed, Docker commands such as `docker ps` and `docker run` became available.

Challenge 1: Docker Image Pull Failure Due to Storage

When attempting to pull the application Docker image from ECR:

```
bash
docker pull 782411432642.dkr.ecr.us-east-1.amazonaws.com/covidrisk:latest
```

The following error occurred:

```
write /var/lib/docker/tmp/GetImageBlobXXXX: no space left on device
```

Solution

The root partition size was increased from 6.8 GB to 15 GB. The commands involved resizing the partition and filesystem:

```
bash
sudo growpart /dev/nvme0n1 1
sudo resize2fs /dev/nvme0n1p1
df -h # Verify the new size
```

After resizing, the available space on the root partition increased to 8.3 GB, enough to pull the Docker image.

Challenge 2: AWS CLI and ECR Authentication

Initially, attempts to login to Amazon ECR using:

```
bash
aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin <registry-
```

failed because `aws` was not installed or configured:

```
Command 'aws' not found
```

Solution

Installed AWS CLI v2 manually:

```
bash
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
sudo apt install unzip
unzip awscliv2.zip
sudo ./aws/install
aws --version # Verify installation
```

Then, exported credentials as environment variables:

```
bash
export AWS_ACCESS_KEY_ID=<your-access-key>
export AWS_SECRET_ACCESS_KEY=<your-secret-key>
export AWS_REGION=us-east-1
```

After this, logging into ECR succeeded:

```
bash
```

```
aws ecr get-login-password --region $AWS_REGION | docker login --username AWS --password-stdin <reg
```

Challenge 3: Running the Docker Container

After pulling the image successfully, running the container faced a naming conflict:

```
docker: Error response from daemon: Conflict. The container name "/covidrisk" is already in use
```

Solution

Removed the old container before running the new one:

```
bash

docker rm -f covidrisk
docker run -d -p 8080:8080 --name covidrisk \
-e AWS_ACCESS_KEY_ID=$AWS_ACCESS_KEY_ID \
-e AWS_SECRET_ACCESS_KEY=$AWS_SECRET_ACCESS_KEY \
-e AWS_REGION=$AWS_REGION \
782411432642.dkr.ecr.us-east-1.amazonaws.com/covidrisk:latest
```

Checking the container status:

```
bash

docker ps
```

The container was then successfully running, accessible via:

```
http://<EC2-public-IP>:8080
```

CI/CD Automation with GitHub Actions

A GitHub Actions workflow was created to automate the process of building, tagging, and pushing the Docker image to Amazon ECR whenever the main branch was updated. Key steps included:

- Installing utilities (jq, unzip)
- Configuring AWS credentials via `aws-actions/configure-aws-credentials`
- Logging into Amazon ECR
- Building and pushing the Docker image
- Deploying the image on EC2

This automation significantly simplified deployments and ensured consistency.

Lessons Learned

Root storage size matters: Docker images can be large, and EC2 default root partitions are often too small for modern apps. Resizing partitions and filesystem is essential.

ECR authentication requires correct credentials: AWS CLI must be installed and properly configured.

Container conflicts: Always ensure old containers are removed before starting a new instance.

CI/CD integration: Automating build and push steps reduces manual errors and ensures the latest code is always deployed.

Conclusion

Deploying a Dockerized application on AWS can involve multiple challenges, especially related to storage, authentication, and container management. By methodically addressing each problem — increasing root partition size, configuring AWS CLI, handling Docker conflicts, and setting up CI/CD pipelines — the application was successfully deployed and accessible to users.

This process provides a clear blueprint for deploying similar applications in the cloud, highlighting the importance of planning for storage, authentication, and automation.